

CP Handbook (Mini)

DSU, MST, Knapsack

Contents

Disjoint Set Union / MST

DSU + Kruskal MST 1

Knapsack

Knapsack (recursive / memoized) 1
Knapsack (iterative table) 2
Knapsack (value-indexed) 2

Disjoint Set Union / MST

DSU + Kruskal MST

DSU_MST/dsu_mst.cpp

Strategy: DSU: path compression + union by rank. MST: sort edges by weight, union endpoints if different components.

Reuse: Connectivity, cycle detection, MST (Kruskal), offline connectivity queries.

```
#include <bits/stdc++.h>
using namespace std;
```

```
// DSU is a Disjoint Set Union (União de Conjuntos Disjuntos).
// É usado para encontrar a que componentes conexos um nodo pertence.
```

```
struct DSU {
    int n;
    vector<int> parent, rank;

    DSU(int N) {
        this->n = N;
        parent.resize(N);

        // Fills the parent vector with 0 incremented 1 by 1.
        iota(parent.begin(), parent.end(), 0);
        rank.resize(N);
    }

    int find(int tgt) {
        if(parent[tgt] == tgt)
            return tgt;
        return parent[tgt] = find(parent[tgt]);
    }

    void join(int c1, int c2) {
        c1 = find(c1);
        c2 = find(c2);

        if (c1 == c2)
            return;

        if(rank[c1] > rank[c2])
            swap(c1, c2);

        parent[c1] = c2;
        rank[c2] += (rank[c1] == rank[c2]);
    }
};
```

```
struct Edge {
    int u, v, w;

    Edge(int U = 0, int V = 0, int W = 0) {
        this->u = U;
        this->v = V;
        this->w = W;
    }

    bool operator< (Edge another) const {
        return this->w < another.w;
    }
};
```

```
int main (void) {
    int n, m;
    cin >> n >> m;

    vector<Edge> edges(m);

    for(int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.push_back(Edge(u, v ,w));
    }

    sort(edges.begin(), edges.end());

    DSU dsu(n);

    vector<Edge> MST;
    for(Edge e : edges) {
        if(dsu.find(e.u) != dsu.find(e.v)) {
            MST.push_back(e);
        }
    }
}
```

```
        dsu.join(e.u, e.v);
    }
}

return EXIT_SUCCESS;
}
```

Knapsack

Knapsack (recursive / memoized)

Dynamic_Programming/knapsack_rec.cpp

Strategy: Top-down dp[nitems][capacity] = max(taken, not_taken). O(N*W).

Reuse: 0/1 knapsack; small N and W.

```
/*
    N itens from 1..N (i);
    Each item has a weight Wi and a value Vi.
    You have a bag with a specific capacity of weight (W),
    meaning that the sum of the chosen items must be at max W.

    What is the biggest possible sum of item values that you can put
    in the bag?
*/
```

```
/*
    The idea is to make a table in the following format:
    Columns: 0 to W;
    Rows: 0 to N;
    Each row is indexed with the number of items available.
    Each column is indexed with the capacity of the bag.
    Each w,v pair is aiming at the row below it.
    E.g W = 15, N = 6:
```

```
    0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(w=6,v=5) 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(w=5,v=6) 1 0 0 0 0 0 0 5 5 5 5 5 5 5 5 5
(w=6,v=4) 2 0 0 0 0 0 6 6 6 6 6 11 11 11 11 11
(w=6,v=6) 3 0 0 0 0 0 6 6 6 6 6 6 11 11 11 11
(w=3,v=5) 4 0 0 0 0 0 6 6 6 6 6 6 12 12 12 12
(w=7,v=2) 5 0 0 0 5 5 6 6 6 11 11 11 12 12 12 17 17
           6 0 0 0 5 5 6 6 6 11 11 11 12 12 12 17 17
```

```
*/
#include <bits/stdc++.h>
using namespace std;
```

```
#define ll long long
```

```
const int MAXN = 10;
const int MAXW = 100005;
```

```
int weight[MAXN], value[MAXN];
ll dp[MAXN][MAXW];
```

```
ll knapsack(int nitems, int capacity) {
    if (nitems == 0 || capacity == 0) return 0;

    if(dp[nitems][capacity] != 0) return dp[nitems][capacity];

    ll not_taken = knapsack(nitems-1, capacity);
    ll taken = 0;

    if(capacity >= weight[nitems] )
        taken = knapsack(nitems-1, capacity - weight[nitems]) + value[
            ↪ nitems];

    dp[nitems][capacity] = max(taken, not_taken);

    return dp[nitems][capacity];
}
```

```
int main(void) {
    ios::sync_with_stdio();
    cin.tie(NULL);
    cout.tie(NULL);
    int nitems, capacity;
    cin >> nitems >> capacity;

    for (int item = 1; item <= nitems; item++) {
        cin >> weight[item] >> value[item];
    }

    cout << knapsack(nitems, capacity) << "\n";
}
```

Knapsack (iterative table)

Dynamic_Programming/knapsack_iter.cpp

Strategy: Bottom-up dp[i][w]; row i built from row i-1. $O(N*W)$.

Reuse: 0/1 knapsack when W small; can be compressed to 1D.

```
/*
    N items from 1..N (i);
    Each item has a weight Wi and a value Vi.
    You have a bag with a specific capacity of weight (W),
    meaning that the sum of the chosen items must be at max W.

    What is the biggest possible sum of item values that you can put
    in the bag?
*/

/*
    The idea is to make a table in the following format:
    Columns: 0 to W;
    Rows: 0 to N;
    Each row is indexed with the number of items available.
    Each column is indexed with the capacity of the bag.
    Each w,v pair is aiming at the row below it.
    E.g W = 15, N = 6:
        0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
(w=6,v=5) 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
(w=5,v=6) 1 0 0 0 0 0 0 5 5 5 5 5 5 5 5 5
(w=6,v=4) 2 0 0 0 0 0 6 6 6 6 6 6 11 11 11 11
(w=6,v=6) 3 0 0 0 0 0 6 6 6 6 6 6 11 11 11 11
(w=3,v=5) 4 0 0 0 0 0 6 6 6 6 6 6 12 12 12 12
(w=7,v=2) 5 0 0 0 5 5 6 6 6 11 11 11 12 12 12 17 17
        6 0 0 0 5 5 6 6 6 11 11 11 12 12 12 17 17
*/

#include <bits/stdc++.h>

using namespace std;
#define ll long long

const int MAXN = 10;
const int MAXW = 100005;

int weight[MAXN], value[MAXN];
ll dp[MAXN][MAXW];

int main(void) {
    int nitems, capacity;
    cin >> nitems >> capacity;

    for(int item = 1; item <= nitems; item++)
        cin >> weight[item] >> value[item];

    for(int item = 1; item <= nitems; item++) {
        for(int w = 0; w <= capacity; w++) {
            dp[item][w] = dp[item-1][w];

            if (w >= weight[item] ) {
                ll taken = dp[item][w];
                ll not_taken = dp[item-1][w - weight[item]] + value[item]
                ↪ ];

                dp[item][w] = max(taken, not_taken);
            }
        }
    }

    cout << dp[nitems][capacity] << "\n";
}
```

Knapsack (value-indexed)

Dynamic_Programming/knapsack_opt.cpp

Strategy: dp[value] = min weight to reach that value; scan sums descending until weight fits.

Reuse: 0/1 knapsack when values small but W huge.

```
/*
    Knapsack problem is optimized here to fit better in memory instead of
    N * W;
    We change the columns to be the sum of values we already achieved
    ↪ instead of the backpack capacity.
    For each value I achieved in my bag, what is the least weight I
    ↪ achieved.
    I = INF
        0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
(w=6,v=5) 0 0 I I I I I I I I I I I I I I I I I I
(w=5,v=6) 1 0 I I I I 6 I I I I I I I I I I I I
(w=6,v=4) 2 0 I I I I 6 5 I I I I 11 I I I I I I
(w=6,v=6) 3
(w=3,v=5) 4
(w=7,v=2) 5
        6

    Feature, Standard Knapsack (Code 2), Value-based Knapsack (Code 1)
    DP State, dp[weight] = max_value, dp[value] = min_weight
    Complexity,  $O(N * W)$ ,  $O(N * \text{Sum of Values})$ 
    Best Used When, "Capacity W is small (105), but Values are huge.", "Values
    ↪ are small (105), but Capacity W is huge."
    Initialization, 0 (base case is zero value), Infinity (base case is
    ↪ infinite weight)
*/

#include <bits/stdc++.h>
#define ll long long
using namespace std;
```